

Webhooks let your app receive real-time notifications for events that happen in PrescribeRx — an encounter is created, an order ships, lab results arrive, a subscription renews. Subscribe to the events you care about and we'll POST a signed JSON payload to your URL when each event fires. This page covers the 27 subscribable events, how to set up a subscription, signature verification, and how to handle failures.

### Two ways to manage webhook subscriptions:

1. **UI** — Sales orgs and clients can self-serve from their portal: `/sales-organization/integrations/webhooks` or `/client/settings/webhooks`. Admins use `/admin/webhooks`.
2. **API** — Programmatic CRUD via `/api/v1/webhooks` (requires the `webhook:read / create / update / delete` abilities, which sales-org and client tokens carry by default — see the [token matrix](#)).

## Subscribable events

Every event is sent as a POST with a JSON body keyed by `event`, `timestamp`, `webhook_id`, `subscription_id`, and `data`. Subscribe to any subset of these on a single subscription, or use a wildcard like `encounter.*` to receive all encounter events.

**All event slugs below are live.** As of 2026-05-01 every category — encounter, order, fulfillment, lab, prescription, subscription, patient, appointment — emits deliveries to matching subscriptions. The slugs you see here are the full public contract; if you don't see expected traffic, double-check your subscription's events array and `is_active` flag (see "Common issues" below).

### Encounters 7 events

`encounter.created`  
Encounter Created

`encounter.assigned`  
Provider Assigned

`encounter.completed`  
Encounter Completed

`encounter.scheduled`  
Encounter Scheduled

`encounter.status_changed`  
Encounter Status Changed

`encounter.prescribed`  
Prescription Written

`encounter.cancelled`  
Encounter Cancelled

### Orders 4 events

`order.placed`

`order.paid`

Order Placed

`order.cancelled`

Order Cancelled

Order Payment Captured

`order.status_changed`

Order Status Changed

## Fulfillment 3 events

`fulfillment.shipped`

Shipment Shipped

`fulfillment.cancelled`

Fulfillment Cancelled

`fulfillment.delivered`

Shipment Delivered

## Labs 4 events

`lab.order_created`

Lab Order Created

`lab.review_completed`

Lab Review Completed

`lab.results_received`

Lab Results Received

`lab.deviation_flagged`

Lab Deviation Flagged

## Prescriptions 1 events

`prescription.written`

Prescription Written

## Subscriptions 4 events

`subscription.created`

Subscription Created

`subscription.cancelled`

Subscription Cancelled

`subscription.renewed`

Subscription Renewed

`subscription.payment_failed`

Subscription Payment Failed

## Patients 1 events

`patient.created`

Patient Created

## Appointments 3 events

`appointment.scheduled`

Appointment Scheduled

`appointment.rescheduled`

Appointment Rescheduled

`appointment.cancelled`

Appointment Cancelled

# Setting up a subscription (5-minute walkthrough)

1

## Create your endpoint

Build a POST endpoint on your server that accepts JSON. Return 2xx within 30 seconds — anything else is treated as a failure and triggers retries.

2

## Subscribe via UI or API

**UI:** Go to your portal's webhooks page, click "Add subscription", paste the URL, pick events, save. The signing secret is shown once — copy it.

**API:**

```
curl -X POST "https://demo.prescribe-rx.com/api/v1/webhooks" \
-H "Authorization: Bearer YOUR_TOKEN" \
-H "Content-Type: application/json" \
-d '{ "url": "https://your-app.com/webhooks/prescriberx", "events": ["encounter.prescribed", "order.placed", "fulfillment.shipped"], "is_active": true }'
```

Response includes id, signing\_secret (shown once — store it securely), and the events you subscribed to.

3

## Verify the signature on every delivery

Every webhook arrives with an X-PrescribeRx-Signature header. The value is sha256=<hex> where <hex> is the lowercase hex digest of HMAC-SHA256(raw\_request\_body, signing\_secret). Compute it on your side using the raw request body bytes (not parsed JSON) and the signing secret you copied at subscription creation. Reject deliveries that don't match.

**Format.** Both production deliveries and test deliveries (from POST /api/v1/webhooks/{id}/test) send the same shape: X-PrescribeRx-Signature: sha256=a1b2c3.... Strip the sha256= prefix before comparing the remainder against your computed HMAC. The prefix exists for forward compatibility (future algorithms will use the same envelope, e.g. sha512=).

```
// PHP $received = $_SERVER['HTTP_X_PRESCRIBERX_SIGNATURE'] ?? ''; $received =
str_starts_with($received, 'sha256=') ? substr($received, 7) : $received; $expected =
hash_hmac('sha256', $rawBody, $signingSecret); if (! hash_equals($expected, $received)) {
http_response_code(401); exit; }
```

```
// Node.js const crypto = require('crypto'); let received = req.headers['x-prescriberx-
signature'] || ''; if (received.startsWith('sha256=')) received = received.slice(7); const
expected = crypto.createHmac('sha256', signingSecret).update(rawBody).digest('hex'); if (!
crypto.timingSafeEqual(Buffer.from(expected), Buffer.from(received))) { return
res.status(401).end(); }
```

**Other delivery headers** — useful for logging, idempotency, and dashboards (sent on every delivery, including tests):

| Header     | Value                   |
|------------|-------------------------|
| User-Agent | PrescribeRx-Webhook/1.0 |

| Header                  | Value                                                                                           |
|-------------------------|-------------------------------------------------------------------------------------------------|
| X-Webhook-ID            | UUID — unique per delivery; use as idempotency key                                              |
| X-Webhook-Event         | The event slug (e.g. <code>order.placed</code> )                                                |
| X-Webhook-Timestamp     | ISO-8601 timestamp matching the body's <code>timestamp</code> field                             |
| X-PrescribeRx-Signature | <code>sha256=&lt;hex&gt;</code> — HMAC-SHA256 of the raw request body using your signing secret |

4

## Test before going live

Trigger a test delivery with the `webhook.test` event:

```
curl -X POST "https://demo.prescribe-rx.com/api/v1/webhooks/{subscription_id}/test" \
-H "Authorization: Bearer YOUR_TOKEN"
```

Watch the delivery logs at `/api/v1/webhooks/{subscription_id}/delivery-logs` or in the UI to see request + response details.

5

## Handle retries gracefully

Failed deliveries (non-2xx, timeout, connection error) are retried up to **3 total attempts** with exponential backoff: `~10s` after attempt 1, `~100s` (`~1.7 min`) after attempt 2, then no further attempts. Each attempt has a **30-second timeout**.

After **50 consecutive failures** across deliveries the subscription is auto-disabled — the UI shows a banner and you'll receive an email; re-enable via `POST /api/v1/webhooks/{id}/re-enable` after fixing your endpoint.

Treat each delivery as **at-least-once**: include an idempotency check keyed on the `webhook_id` field (a UUID generated per delivery, also sent in the `X-Webhook-ID` header).

## Subscription management endpoints

| Verb | Path                                      | Purpose                           | Required ability            |
|------|-------------------------------------------|-----------------------------------|-----------------------------|
| GET  | <code>/api/v1/webhooks/event-types</code> | List subscribable event types     | <code>webhook:read</code>   |
| GET  | <code>/api/v1/webhooks</code>             | List your subscriptions           | <code>webhook:read</code>   |
| POST | <code>/api/v1/webhooks</code>             | Create a subscription             | <code>webhook:create</code> |
| GET  | <code>/api/v1/webhooks/{id}</code>        | Get one subscription              | <code>webhook:read</code>   |
| PUT  | <code>/api/v1/webhooks/{id}</code>        | Update URL / events / active flag | <code>webhook:update</code> |

| Verb          | Path                                | Purpose                                     | Required ability |
|---------------|-------------------------------------|---------------------------------------------|------------------|
| <b>DELETE</b> | /api/v1/webhooks/{id}               | Delete a subscription                       | webhook:delete   |
| <b>POST</b>   | /api/v1/webhooks/{id}/rotate-secret | Roll the signing secret                     | webhook:update   |
| <b>POST</b>   | /api/v1/webhooks/{id}/re-enable     | Re-enable an auto-disabled subscription     | webhook:update   |
| <b>POST</b>   | /api/v1/webhooks/{id}/test          | Trigger a webhook.test delivery             | webhook:update   |
| <b>GET</b>    | /api/v1/webhooks/{id}/delivery-logs | Recent delivery attempts (request/response) | webhook:read     |

## Sample payload shape

Every delivery follows the same envelope. The data block varies by event type. Below is the actual shape currently emitted for `encounter.prescribed` — the data block contains the four fields shown, no more.

```
{ "event": "encounter.prescribed", "timestamp": "2026-04-30T20:34:12+00:00", "webhook_id": "019ddf8a-7c3d-7234-ab01-c9d8e7f6a5b4", "subscription_id": "019dd4a1-3c2b-7100-89ab-cdef01234567", "data": {
  "encounter_id": "019dd4d8-1a2b-7300-89ab-000111222333", "encounter_number": "ENC-7658107290",
  "patient_chart_id": "019dd4ce-5f6a-7200-89ab-112233445566", "prescribed_at": "2026-04-30T20:34:08+00:00" } }
```

Need richer data (e.g., the prescription itself, the provider's NPI)? Hit `GET /api/v1/encounters/{id}` with your token after receiving the webhook. The webhook is the doorbell; the API is the deeper read.

## Wildcards + filters

Use a wildcard to subscribe to a whole category in one shot:

```
{ "events": ["encounter.*", "order.placed", "order.paid", "fulfillment.*"] }
```

`encounter.*` matches all 7 encounter events. `fulfillment.*` matches all 3 fulfillment events. Use `*` alone to subscribe to everything (not generally recommended — narrower subscriptions are easier to debug).

**Scope:** Subscriptions only fire for events within your token's data scope. A client-token subscription only receives events for that client's encounters/orders/etc; a sales-org-token

subscription receives events for the entire org tree. Cross-tenant events are never delivered regardless of subscription configuration.

## Common issues

| Symptom                                      | Likely cause                                                                                                                                             | Fix                                                                                                                                                                                                               |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| No deliveries arriving                       | Subscription <code>is_active=false</code> , or auto-disabled after 50 failures, or no events of the subscribed slug have been emitted yet on your tenant | Check status via <code>GET /webhooks/{id}</code> , re-enable if needed; trigger one of your subscribed events (e.g. POST a unified intake to fire <code>encounter.created</code> + <code>patient.created</code> ) |
| Signature mismatch                           | Verifying parsed JSON instead of raw bytes, or forgetting to strip the <code>sha256=</code> prefix from the header value before comparing                | HMAC the raw request body bytes. Strip the <code>sha256=</code> prefix from <code>X-PrescribeRx-Signature</code> before comparing the remainder.                                                                  |
| Webhook called multiple times for same event | Your endpoint returned non-2xx → retry, OR delivery succeeded but your acknowledgement timed out                                                         | Idempotency check keyed on <code>webhook_id</code> (also in <code>X-Webhook-ID</code> header)                                                                                                                     |
| Slow delivery                                | Endpoint >30s response triggers timeout retry                                                                                                            | Acknowledge with 200 immediately, process async                                                                                                                                                                   |
| Subscription URL not saving                  | Localhost / 192.x.x.x / 10.x.x.x rejected                                                                                                                | Use a public-routable URL; ngrok works for dev                                                                                                                                                                    |